SPECIAL TOPICS IN DATA SCIENCE — BUILDING LLM SYSTEMS: FROM MODELS TO PRODUCTION

Project Proposal: Resume & Job Description Extractor

Repository: [https://github.com/hamza-alkaabneh/resume-jd-extractor-llm]

1. Team and roles

Phase 1 Role	Team Member Name	Phase 1 Responsibilities
Product Lead	Hamza Mohammed Alkaabneh	Recruiter user research, problem definition, and user interview evidence.
Data Lead	Hamza Mohammed Alkaabneh	Collecting, anonymizing, and documenting the CV/JD dataset and Data Card.
Evaluation Lead	Hamza Mohammed Alkaabneh	Authoring the 30-row seed test set, expected answers, and scoring criteria.
Engineering Lead	Hamza Mohammed Alkaabneh	Repository setup, baseline pipeline design, and free-compute feasibility check.

2. User and problem

The target users are technical recruiters and HR screening coordinators who evaluate entry-level Computer Science and Data Science candidates. Today, when screening dozens of PDF resumes against specific Job Descriptions (JDs), recruiters manually read inconsistently formatted multi-column CVs and copy candidate qualifications into tracking spreadsheets. This manual workflow takes 3 to 5 minutes per resume and frequently causes reviewers to overlook qualified applicants who express skills using different terminology.

Evidence of user need: Technical hiring coordinators usually have their primary pain point as verifying strict eligibility constraints such as graduation date, minimum cumulative GPA, total months of relevant work experience, and mandatory frameworks across varied PDF layouts. An automated tool capable of extracting structured fields and answering candidate screening questions with exact document citations would cut screening time by more than half.

3. What the system will do

The system ingests anonymized candidate resumes (PDF/text) and structured Job Descriptions, extracts core qualifications into a standardized schema, and answers recruiter questions about candidate eligibility and skill gaps. Every response cites the exact source file and section while refusing to guess unstated personal attributes.

- **Interaction 1 (Direct Extraction):** *User:* "What are Candidate CV_01's GPA, graduation date, and Python frameworks?" → *System:* "Candidate CV_01 has a GPA of 3.72/4.0, graduates in June 2026, and lists FastAPI, Scikit-Learn, Pandas, NumPy, and TensorFlow (Source: CV_01.pdf, p.1)."
- **Interaction 2 (Constraint Matching):** *User:* "Does Candidate CV_02 qualify for the Backend Software Engineer role (JD_03)?" → *System:* "Yes; CV_02 holds a B.Sc. in Software Engineering, 14 months of Node.js backend experience (exceeding the 12-month requirement), Docker, and PostgreSQL (Sources: CV_02.pdf p.1, JD_03.txt)."
- **Interaction 3 (Refusal / Guardrail):** *User:* "What is Candidate CV_01's phone number and home address?" → *System:* "I don't know. All personal contact information and addresses have been anonymized and removed from the dataset (Source: CV_01.pdf)."

4. Data

The dataset contains 60 anonymized technical resumes and 15 technical Job Descriptions (75 documents total, ~90 pages, ~31,500 tokens) in English and bilingual Arabic/English formats. Documents are sourced from the public CC0 Kaggle Updated Resume Dataset, Hugging Face Job Descriptions dataset, and synthetic student CVs modeled on local university templates. All personally identifiable information (names, phone numbers, emails, addresses) has been scrubbed and replaced with neutral identifiers (`CV_01`–`CV_60`). Five complete sample resumes (`CV_01`–`CV_05`) and three Job Descriptions (`JD_01`–`JD_03`).

5. Planned architecture(first version)

> **Phase 2 Baseline Pipeline:**
> User Question + CV/JD Docs → PyMuPDF Layout Parser & Chunker → BGE-M3 Embeddings + FAISS Index → Top-k Retriever → Open-Weights LLM (Llama-3.1-8B-Instruct via free API/Colab) → Grounded Answer with Source Citation

This architecture fits entirely within free Google Colab T4 GPUs and free Hugging Face / NVIDIA NIM inference endpoints. In Phase 3, the problem supports combining two core course techniques: **Hybrid RAG** (BM25 lexical search + dense embeddings with cross-encoder reranking) alongside a **Tool-Calling Agent** (for exact date/month arithmetic and hard constraint verification) or **QLoRA Fine-Tuning** for strict JSON schema extraction.

6. How success would be measured

An answer is considered correct only if it matches the ground-truth fact, cites the right document, and abstains when data is missing. Across every phase release, there would be a report of three quantitative metrics on the test set:

- **Answer Accuracy (Token F1 & Exact/Semantic Match):** Share of test questions whose extracted values or eligibility verdicts match the pre-written ground truth.
- **Citation Precision:** Percentage of responses that cite the exact source document (`CV_xx.pdf` or `JD_xx.txt`) specified in the test set.
- **Refusal / Abstention Accuracy:** Percentage of unanswerable or privacy-sensitive questions where the model properly outputs "I don't know" instead of hallucinating.

7. Risks

- **PDF Parsing Noise (Data Risk):** Complex multi-column PDF layouts can scramble section headers. *Backup Plan:* Use block-aware parsing via `PyMuPDF` and maintain clean text mirrors so retrieval evaluation is never blocked by OCR artifacts.
- **Free Compute Quotas (Compute Risk):** Rate limits on free APIs or Colab GPU timeouts during evaluation runs. *Backup Plan:* Cache document embeddings locally in FAISS and use 4-bit quantized models (`Qwen2.5-7B-Instruct` / `Llama-3.1-8B`) that run comfortably on a single free T4 GPU.
- **Scope Creep (Scope Risk):** Spending time on frontend UI instead of retrieval and extraction quality. *Backup Plan:* Restrict the interface to a minimal command-line evaluation script and focus engineering effort strictly on beating baseline test-set scores.